Abalone-Game

Aditya Phopale, Durganshu Mishra, and Gaurav Gokhale

TUM School of Computation, Information and Technology, Technical University of Munich

November 12, 2023

1 Introduction

We implemented our own search strategy and optimizations for determining player moves in the two player board game Abalone. Our implementation is based on the NegaMax with Alpha-Beta pruning search algorithm. The code is OpenMP enabled. Besides gaining a significant performance gain through parallelization, we also experimented other optimizations including Principal Variation (with and without move caching), iterative deepening and transposition table.

2 Alpha Beta Algorithm

Alpha-beta pruning is a search algorithm used to reduce the number of nodes that need to be evaluated in the search tree of the MiniMax algorithm. Our algorithm is based on NegaMax [1], which is a simplified version of MiniMax. We can simplify the MiniMax further by removing the distinction between maximizing and minimizing nodes. By simply negating the result returned from the recursive call, each node can be treated as a maximizing node. However, we need to make another modification to achieve the same result as the MiniMax algorithm. In a leaf node, the evaluate() function should return the score from the perspective of the player making the move.

For example, let's consider a leaf node that has a score of -6 in the original minimax scheme. This score indicates that the position is favorable for player 2. In the new scheme, if it's player 1's turn to move, a score of -6 should be returned. However, if it's player 2's turn to move, then a score of 6 would be returned.

AlphaBeta is an extension of the NegaMax algorithm. It stops evaluating a move when it finds at least one possibility that proves the move to be worse than a previously examined move. Such moves are not further evaluated. When applied to a standard minimax tree, AlphaBeta produces the same move as the minimax algorithm but eliminates branches that cannot affect the final decision.

During the traversal of the game tree, as the algorithm explores different moves and their resulting game states, it updates the alpha and beta values accordingly. For the maximizing player, the alpha value is updated with the maximum of the current alpha and the evaluated move's score. For the minimizing player, the beta value is updated with the minimum of the current beta and the evaluated move's score.

3 Optimizations

3.1 Iterative Deepening

Instead of tackling the entire tree at once, it can be advantageous to determine the tree value and principal variation in steps, depending on the specific application. This approach involves iteratively deepening the search process [2]. Initially, the game tree value and principal variation are determined for a tree of depth one. Then, a similar process is carried out on a tree of depth two, and this iterative deepening continues until the desired depth of the tree has been searched.

There are two advantages to this scheme. Firstly, in situations where there is a time constraint on the search, attempting to search the entire tree in a single pass may be too time-consuming. However, by conducting the search in steps, even if the search needs to be stopped at a certain depth, the previous iterations provide a reasonable solution, albeit at a lower search depth. Secondly, each iteration gathers valuable information

about the tree that can be used in subsequent iterations, enhancing the efficiency and effectiveness of the search process.

This approach was not used since it did not provide much speedup in the parallel version.

3.2 Transposition Table

A transposition table, also known as a hash table, is a data structure used in game-playing algorithms, including those based on the minimax algorithm with alpha-beta pruning. It serves as a cache that stores previously computed game positions along with their associated scores or other relevant information.

This approach was thought of to be especially useful when facing a draw situation in endgames where moves would be repeated until timeout. Getting the moves from the cache without searching them then would provide much more benefit. However, the parallel version was leading to some data races which we couldn't debug in time. So this approach was not used.

3.3 PV-Splitting

In a perfectly ordered tree, the alpha-beta algorithm searches the first sequence of moves that it encounters, known as the principal variation (PV) [3]. At each node along the PV, the algorithm establishes a high lower bound, also referred to as an alpha value. This lower bound represents the minimum score that the maximizing player is guaranteed in the current branch.

By searching through the principal variation right from the start, the search benefits from this high lower bound. As the algorithm explores subsequent branches and nodes, it can use the established alpha value to prune away irrelevant branches early in the search. Any moves that fall below the alpha value can be disregarded since they cannot possibly improve the outcome for the maximizing player.

This property of the alpha-beta algorithm allows for efficient pruning and significantly reduces the number of nodes that need to be evaluated, leading to faster search times. By focusing on the principal variation and leveraging the established lower bound, the algorithm can quickly identify promising moves and disregard less favorable ones, improving its overall search efficiency.

In the PVSplit algorithm, the search begins with exploring the first branch at a PV node before parallel searching of the remaining branches can start. Each processor travels down the first branch at each PV node until reaching the PV node one level above the leaf nodes. At this point, one processor searches the first branch while the other processors wait. Once the first branch has been examined, all processors join the search effort. Each processor takes one branch at a time and determines its value. If a processor discovers an improvement to the node's score, it informs the other processors of the updated value. When there are no unassigned branches left, a processor that runs out of work waits until the other processors finish. After all branches have been examined, the search effort moves to the current node's parent. Since the value of the parent node's first branch was already computed, parallel search can also start at the parent. This process continues upwards in the tree until all branches at the root node have been examined.

This approach significantly reduced the parallelization overhead. Moreover, for communication of alpha bounds, a common shared array was allocated as per PV depths. The threads at each depth would write their values inside this array if their evaluation was greater than the existing alpha in the array. This update was done inside a critical region to avoid data races.

PV splitting, while aiming to improve efficiency in game tree search and parallelization, has some potential drawbacks. If the game tree is not strongly ordered (where the first branch at any node is the best 70 percent of the time), search overhead increases as parallel search is conducted with a sub-optimal bound [4]. Depending on the branching factor, it may also not be able to use a large number of processors because of the excessive synchronization overhead.

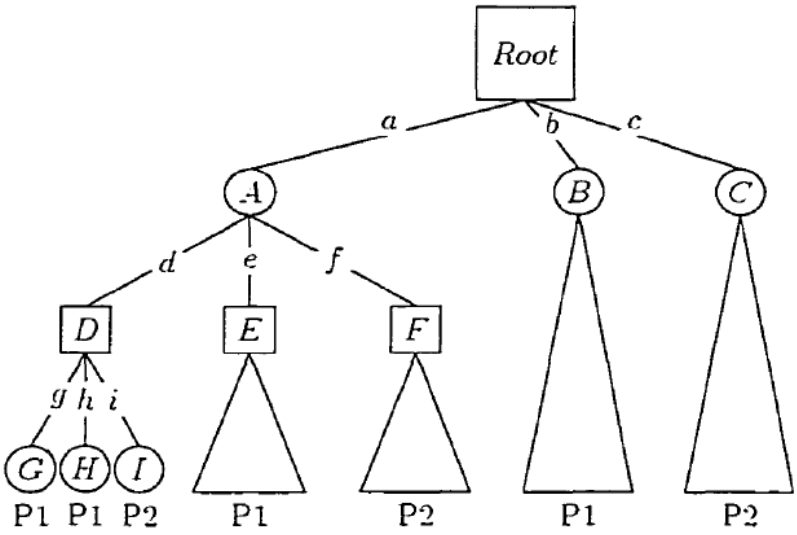


Figure 1 PV splitting using two processors on a small tree [1]

3.4 Death mode

This optimization was heuristic in nature. During the competitive games, it becomes crucial for the player to keep the track of remaining game time. Our code does the same, and automatically adjusts the search depth when only 5 seconds are remaining. For instance, if the program was started with a search depth of 7, the search depth will be reduced to 4 as soon as only 5 seconds of player's game time is left. This allows to get moves faster to avoid loss due to timeout.

These heuristics were identified after repeated runs on several depths as discussed in section 4.

3.5 Parallelization and final setup

We used the OpenMP task construct for parallelism. Each task is assigned a private copy of the board and evaluator, allowing individual threads to work with their own local instances of these classes.

Initially, we attempted parallelization on top level. However, this approach led to load imbalance because some nodes had shallower search depths compared to their siblings. Consequently, nodes had to wait for children with deeper search depths to complete, resulting in idle threads. Additionally, resource and memory usage became difficult to control due to the varying number of playable moves at different tree nodes and depths.

We adopted the concept of PV Splitting to control load balancing and parallelism in the search tree. Parallel tasks are now spawned only on promising nodes that have already been identified through the principal variation. This approach allows for better load balancing and parallel execution by focusing resources on established advantageous positions.

Further, we used the compiler flags `-O3`, `-fno-alias`, and `-xhost` to compile the code on the Intel C++ compiler. These combinations allow further compiler optimizations such as ignoring unproven aliasing hazards, aggressive loop transformations, and allowing the compiler to generate the instructions for the highest instruction set available on the compilation host.

4 Results

4.1 Effect of optimizations

Table 1 compares the run time for different strategies. The results corresponded to a single move with a search depth of 5 and were collected on a single batch node of the SuperMUC. For parallel runs, 48 hardware threads were utilized.

As is evident, sequential AlphaBeta is quite faster than the sequential NegaMax. This is due to the effective alpha-beta pruning that cutoffs unnecessary search branches.

Additionally, OpenMP parallelization with PV splitting further improves the performance. We tried two variants of PV splitting: one by caching PV nodes (to avoid unnecessary move searches) and another without caching. As these results correspond to a single move, caching will not significantly affect the performance. Hence, the runtime for the two variants is in the same order. However, in an actual game, caching improves the move search and performance of the strategy.

Table 2 shows the equivalent figures for the obtained speedup of different strategies.

Table 1 Comparisons of the achieved runtime in seconds with a search depth of 5

Strategy	Start Position (in s)	Midgame1 Position (in s)
NegaMax	410.317	536.78
Alpha-Beta Sequential	1.178	1.751
Alpha-Beta PVSplit without Caching	0.191	0.432
Alpha-Beta PVSplit with Caching	0.140	0.451

Table 2 Comparisons of the achieved speedup with a search depth of 5

Strategy	Speedup Start	Speedup Midgame1
Alpha-Beta Sequential	1	1
Alpha-Beta PVSplit without Caching	6.167	4.0532
Alpha-Beta PVSplit with Caching	8.4143	3.8824

4.2 Effect of parallelization

Majorly, three different costs are associated with the parallelization: Communication cost, Synchronization cost and, Search cost. Communication cost refers to the overhead due to communication of alpha and beta values from one thread to another. Since we're using a shared memory layout, this should be minimal. Synchronization cost refers to the overhead resulting due to some processors waiting idle for other processors to complete the task.

Lastly, the presence of search overhead arises from the fact that the parallel alpha-beta algorithm explores nodes that the sequential version would have skipped. When parallel search begins at a particular node, it's possible that the best score hasn't been found yet. Consequently, parallel search is carried out with a broader range than in the sequential scenario. Additionally, once parallel search is underway, one processor may realize that further search at the node is unnecessary due to a cut-off condition, resulting in the other processors essentially performing redundant work.

Figure 2 shows the performance of PV splitting and caching with varying number of hardware threads on the start position. The speedup increases linearly as we increase the number of threads. However, the trend reverses for more than 20 threads. Our intuition is that it is due to an increase in synchronization overhead. Also, according to Manohararajah [1], PV splitting may not benefit from a large number of processors or threads as the moves may not be ordered (absence of strongly ordered tree).

Additionally, Figure 3 shows the performance of PV splitting with caching for different search depths and start positions on the game board. The results are collected for 48 hardware threads. It can be seen that maximum speedup is obtained for search depth 5. For higher search depths, an increased in task creation and synchronization may be leading to lower performance.

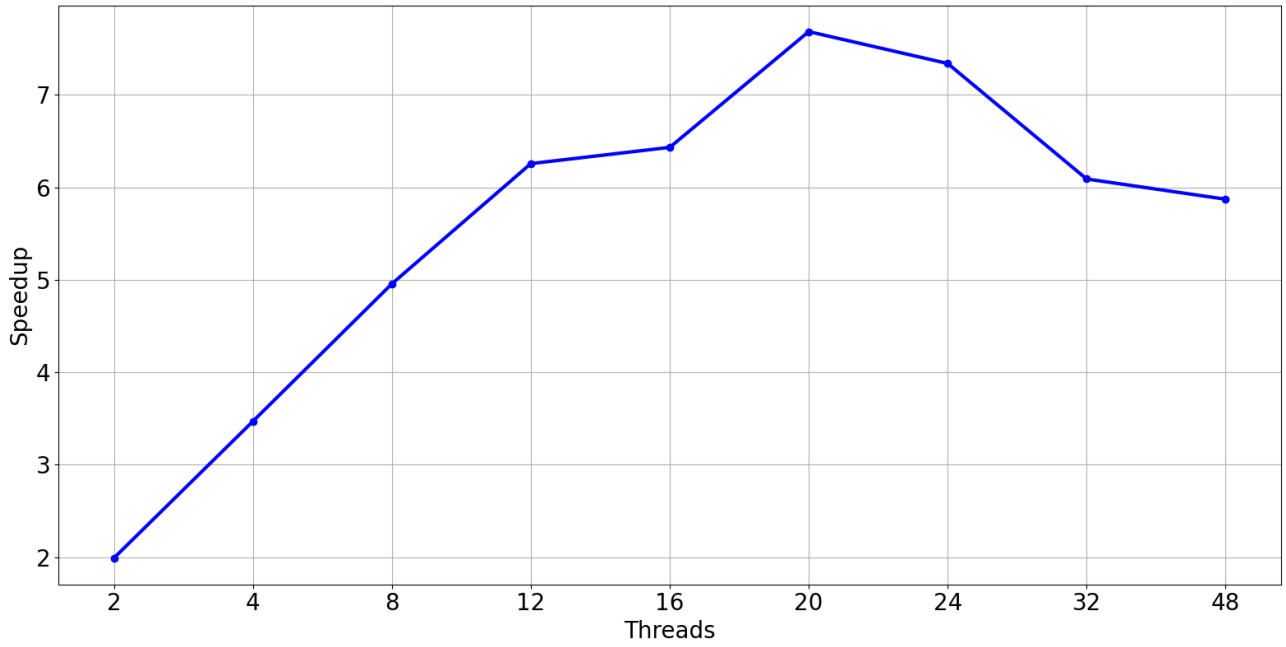


Figure 2 Speedup graph for varying number of threads using PVSplitting with caching

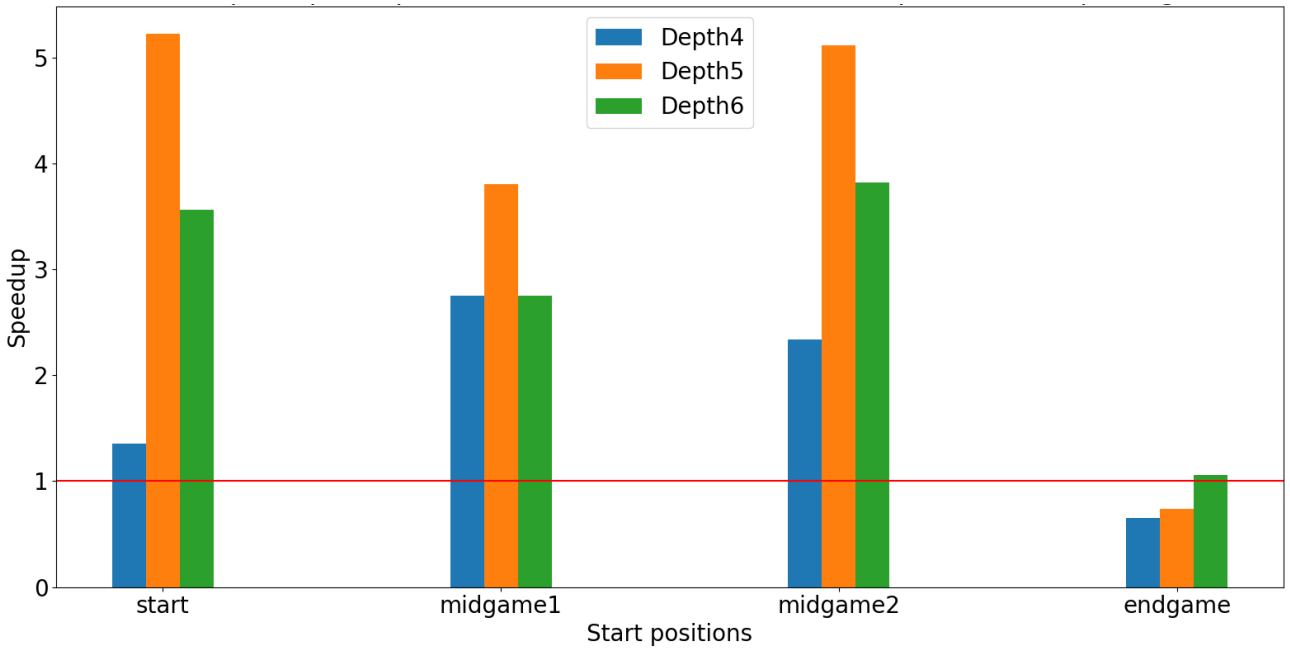
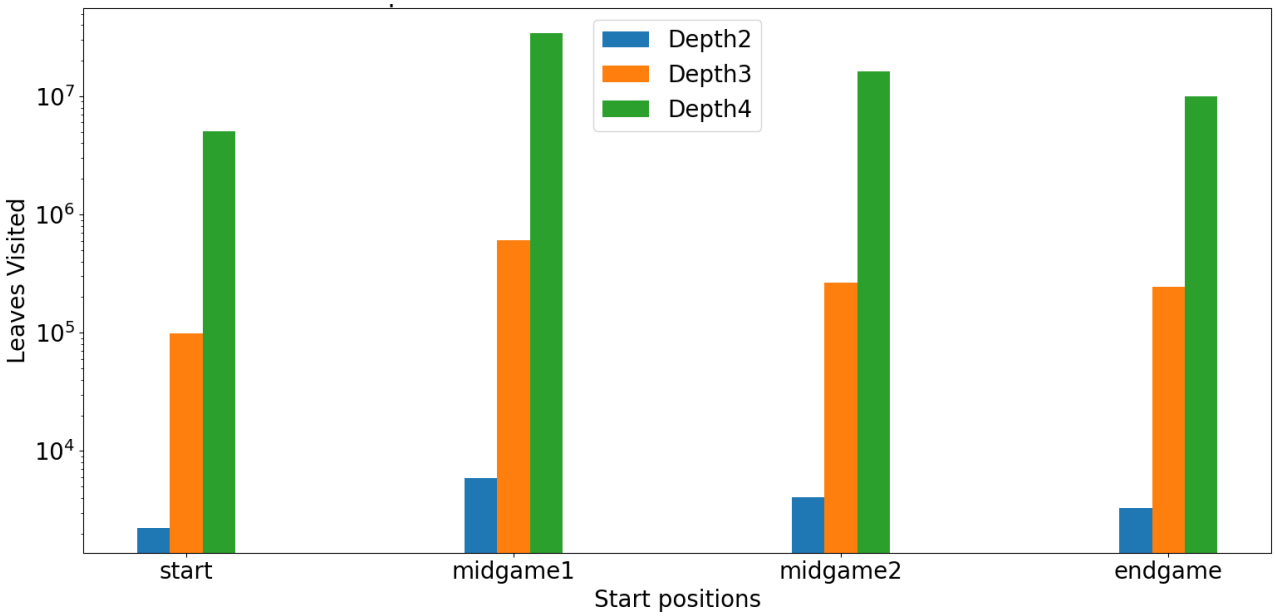


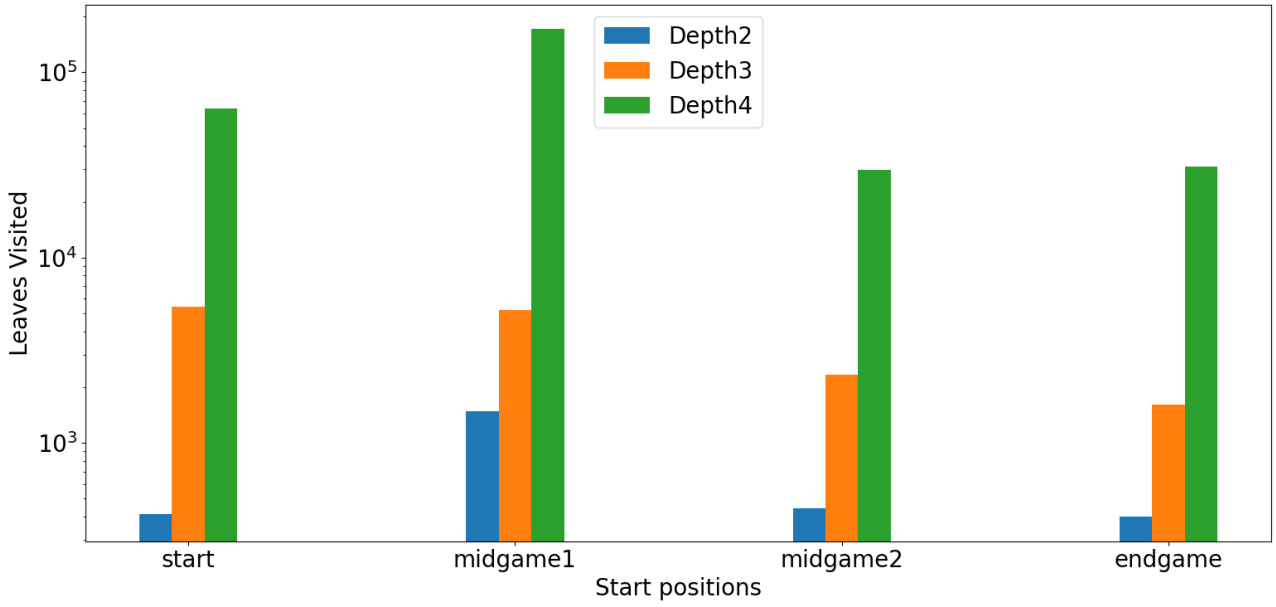
Figure 3 Speedup graph for different search depths and start positions using PVSplitting with caching

4.3 Evaluation Metrics

An important metric to evaluate the performance of a search strategy is by visualizing the number of leave nodes are visited per move. Figure 4a and Figure 4b show the variations of number of leaves visited per move for NegaMax and Alpha-Beta pruning respectively. Effectiveness of Alpha-Beta is clearly reflected as the number of leaves visited have significantly reduced in comparison to Negamax.



(a) Leaves visited for NegaMax



(b) Leaves visited for AlphaBeta with PV splitting and caching

Figure 4 Leaves visited for Negamax and AlphaBeta case

References

[1] V. Manohararajah, “Parallel alpha-beta search on shared memory multiprocessors,” 2001. https://api.semanticscholar.org/CorpusID:54124350?utm_source=wikipedia.

[2] A. Papadopoulos, K. Toumpas, A. Chrysopoulos and P. A. Mitkas, "Exploring optimization strategies in board game Abalone for Alpha-Beta search," 2012 IEEE Conference on Computational Intelligence and Games (CIG), Granada, Spain, 2012, pp. 63-70, doi: 10.1109/CIG.2012.6374139.

[3] Hasan, Khondker, "A Distributed Chess Playing Software System Model Using Dynamic CPU Availability Prediction," 2011. doi: 10.13140/RG.2.1.1053.0729.

[4] T. Marsland and M. Campbell, “Parallel search of strongly ordered game trees,” ACM Computing Surveys, vol. 14, no. 4, pp. 533–551, Dec. 1982, doi: 10.1145/356893.356895.